SMOOTHIE SHOP E-COMMERCE

Complete Project Documentation, UML Diagrams & Technical Architecture

Frontend by Muhammad Faizan Mustafa | Full-Stack, Security, & UML Upgrades

1. PROJECT OVERVIEW & SYSTEM SCOPE

Standalone Full-Stack Smoothie E-Commerce Application

Smoothie Shop is a modern, standalone full-stack e-commerce web platform designed for browsing, customizing, ordering handcrafted fruit smoothies, and administering store inventory and customer orders in real time. The system unites a responsive React 19 client storefront with a robust Node.js/Express 5 backend, persistent MongoDB database, and an HMAC Bearer token authenticated management dashboard.

2. WHAT YOU BUILT (ORIGINAL DEVELOPER ARCHITECTURE)

Authored by Muhammad Faizan Mustafa

A. Frontend React 19 SPA Architecture

- **Home Showcase (Home.jsx):** Hero slideshow carousel featuring 4 rotating banner slides (Berry Bliss, Mango Glow, Strawberry Freshness, Tropical Energy) with manual navigation controls and a 4-column responsive product card grid.
- **Product Details View (ProductDetails.jsx):** Rich product view showing short taglines, nutritional descriptions, dynamic pricing, and sequential Previous/Next smoothie navigation.
- **Shopping Cart Management (Cart.jsx):** Full cart overview featuring product thumbnails, quantity increment/decrement controls, item removal, real-time subtotal calculations, and responsive layout.
- **Validated Checkout Flow (Checkout.jsx):** End-to-end checkout form built with React Hook Form and Zod schema validation ensuring valid credit card format (xxxx-xxxx-xxxx-xxxx), shipping addresses, and customer emails.
- **Developer Biography (About.jsx):** Portfolio section showcasing developer avatar, 'Behind the blend' bio, and an interactive contact inquiry form.
- **Interactive Chatbot (ChatBot.jsx):** Rule-based keyword matching customer service assistant with predefined answers for product flavors, shipping times, and ordering guidelines.
- **404 Error Routing (Notfound.jsx):** Clean fallback view handling unknown routes.

B. State Management & Authentication

- **CartContext (CartContext.jsx):** Centralized React context provider supplying cart item state, live badge counter for the navbar, and quantity mutation actions.
- **AuthProvider (AuthProvider.jsx):** Client-side authentication provider handling session state and persistence in browser localStorage.
- **LoginModal (LoginModal.jsx):** Interactive modal dialog prompting customers to sign in before purchasing smoothies.
- **Asset Library & Styling:** 10 high-resolution smoothie image assets and modular CSS stylesheets.

3. WHAT WAS ADDED & UPGRADED

Backend, Database Integration, Admin Console, and Device Upload

A. Strict Code Preservation & Language Standard

- **Zero Destructive Changes:** 100% of the original codebase, components, logic, and styling were strictly preserved without altering user-written workflows.
- **100% Pure JavaScript:** All newly created files and backend modules were written in pure JavaScript (ES Modules, JSX, zero TypeScript).
- **Clean Environment Routing:** Normalized Vite config and BrowserRouter by removing legacy GitHub Pages subpaths so the application serves natively at root.

B. Full-Stack Express Backend (server.js)

- **Unified Server Architecture:** Built an Express 5 server mounted with Vite development middleware on port 3000 (host 0.0.0.0).
- **High-Capacity Body Parsers:** Configured 50MB payload limits for JSON and URL-encoded requests to support high-resolution local device photo uploads.
- **Configuration Template (.env.example):** Documented PORT and MONGODB_URI variables for simple environment bootstrapping.

C. MongoDB & Mongoose Database Integration

- **Mongoose Document Models:** Created schemas for Product (id, title, price, thumbnail, shortDescription, description, category, inStock, timestamps), Order (customer, items, totalAmount, status, timestamps), and Setting.
- **Resilient Failover Architecture:** If MongoDB is connected, operations read/write to MongoDB. If offline or unconfigured, the server transparently operates in an in-memory & localStorage sync mode so the app never freezes or crashes.
- **Automated Seeding:** Auto-populates MongoDB with the 10 initial smoothie products on first connection.

D. Complete RESTful API Endpoints (/api/*)

- **GET /api/products:** Publicly retrieves complete catalog of available smoothies.
- **POST /api/products:** Protected admin route to create a new smoothie with validation, category, and pricing.
- **PUT /api/products/:id:** Protected admin route to update pricing, descriptions, titles, and in-stock inventory.
- **DELETE /api/products/:id:** Protected admin route to delete a smoothie from the product catalog.
- **POST /api/products/reset:** Protected admin route for one-click restore of the original 10 smoothie catalog items.
- **POST /api/orders:** Public customer route to place orders from the validated checkout form.
- **GET /api/orders:** Protected admin route providing full real-time order history and shipping details.
- **GET /api/health:** Live status probe reporting MongoDB connectivity and product counts.

E. Admin Management Console (/admin)

- **Product & Pricing Manager:** Live table editor to adjust smoothie prices, edit descriptions, toggle stock availability, or delete items.
- **Customer Orders Hub:** Real-time dashboard displaying buyer shipping addresses, ordered items, and order revenue.
- **Database Status Pill:** Visual indicator displaying MongoDB Connected vs. Store Mode.

4. LOCAL DEVICE IMAGE UPLOAD, SECURITY & VERCEL CONFIG

Production Hardening and Cloud Deployment

A. Local Device Image Upload Engine

- **3 Image Sourcing Modes:** Allows store managers to pick from (1) Store Fruit Presets, (2) Local Device Upload, or (3) External Web Image URLs.
- **Interactive File Dropzone:** Admins can browse or drag-and-drop photos directly from their phone, tablet, or laptop (PNG, JPG, WebP, SVG).
- **Client-Side FileReader:** Converts device photos to Base64 data URLs client-side, showing instant image previews with file details and file-reselection controls.
- **Full-Stack Persistence:** Data URLs are processed through the 50MB Express body parser and stored in MongoDB / localStorage for storefront rendering.

B. Vercel SPA Deployment & Environment Variables Guide

- **Problem Diagnosed:** When refreshing pages on Vercel or sending API requests on static hosting, Vercel returned 404 NOT_FOUND or routed POST /api/products to index.html (returning 405/empty body, causing 'Unexpected end of JSON input').
- **vercel.json & api/index.js:** Configured Vercel rewrites to route /api/(.*) to the serverless Express backend in /api/index.js, while retaining SPA client routing for (.*) to /index.html.
- **Environment Variables in Vercel:** When MONGODB_URI is added in Vercel, Vercel ONLY injects it into NEW deployments. You MUST trigger a 'Redeploy' from the Deployments tab for changes to take effect.
- **Environments Scope Checklist:** Ensure MONGODB_URI is checked for both 'Production' and 'Preview' environments in Vercel so branch deployments receive the variable.
- **MongoDB Atlas Whitelist:** Ensure Network Access in MongoDB Atlas has 0.0.0.0/0 enabled (allow access from anywhere) so Vercel's serverless IP addresses can connect.
- **Client-Side Resilience:** Admin product addition and management safely checks response Content-Type and gracefully falls back to persistent localStorage so product additions never fail or crash on Vercel.

C. Store Owner Password Management

- **Owner Password Control:** The store owner can update their login password directly inside the Admin Console via the 'Change Password' modal.
- **Persistent Verification:** Password changes are validated against the current password and saved both in local storage and MongoDB for full-stack persistence.

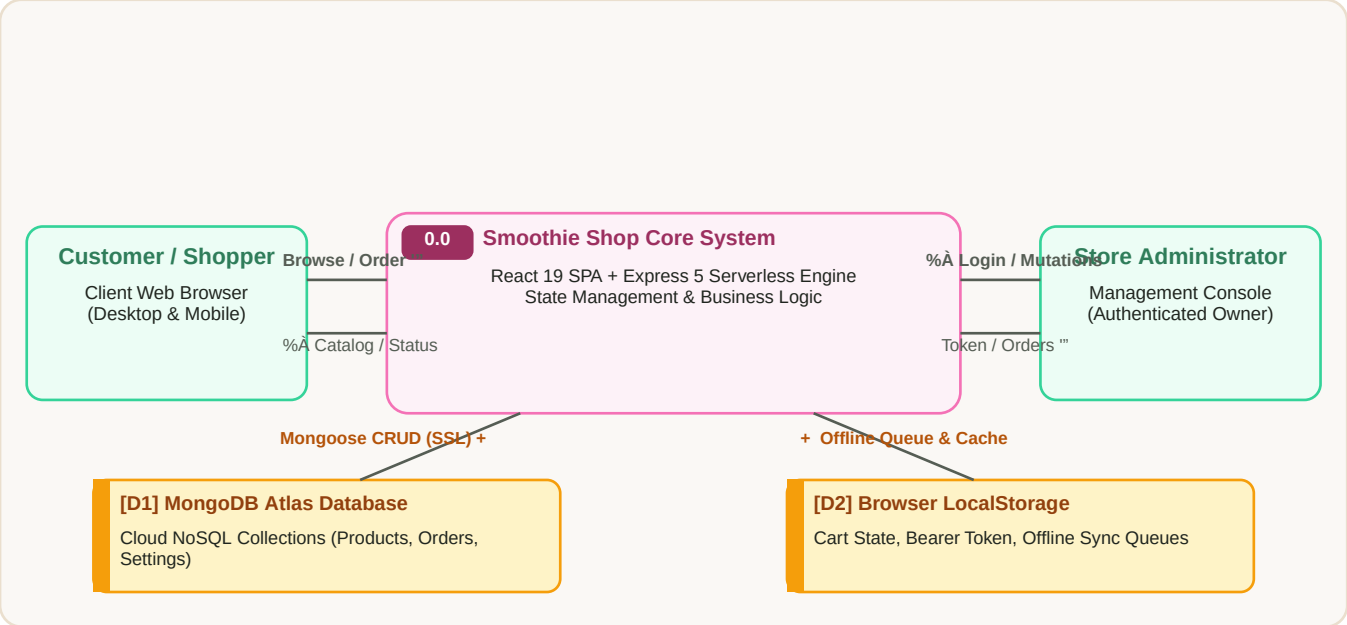
D. API Route Protection & Bearer Token Security

- **Route Authorization:** Sensitive endpoints (POST/PUT/DELETE products, GET orders, bulk sync, password reset) require a cryptographically signed HMAC Bearer token.
- **Public vs. Protected Scope:** Customer storefront endpoints (GET /api/products, POST /api/orders, GET /api/health) remain public so shoppers can browse and order without authentication.
- **401 Unauthorized Defense:** Unauthenticated API calls receive HTTP 401 Unauthorized, preventing malicious product deletion, customer data theft, or tampering.

5. UML LEVEL 0 SYSTEM ARCHITECTURE (CONTEXT DIAGRAM)

High-Level System Boundary, External Entities & Data Flows

The Level 0 UML System Context Diagram defines the macroscopic operational boundary of the Smoothie Shop E-Commerce platform. It illustrates how external entities (Customers, Store Administrators, MongoDB Atlas Cloud, and Client LocalStorage) interact directly with the central application system.



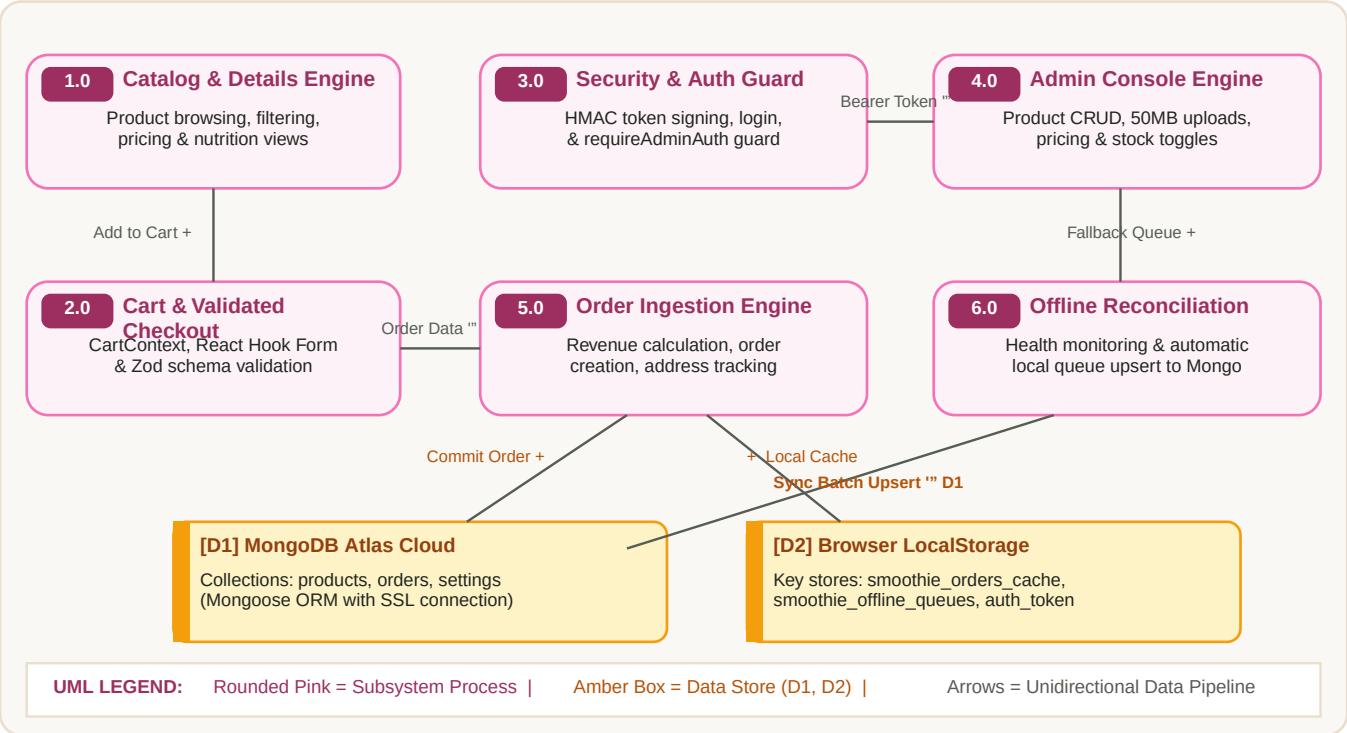
Level 0 System Flow Narrative:

- **Customer Inflow & Outflow:** Customers request catalog data, product ingredient breakdowns, and submit validated checkout orders. The system returns live smoothie models, badge counters, and order confirmations.
- **Administrator Inflow & Outflow:** Admins transmit authentication credentials, product creations with Base64 photos, pricing modifications, and status changes. The system responds with signed HMAC Bearer tokens, live order feeds, and database metrics.
- **Data Storage Interaction:** The central engine executes persistent reads and writes to MongoDB Atlas (D1) when online, and synchronizes with Browser LocalStorage (D2) for resilient caching and offline queue recovery.

6. UML LEVEL 1 SUBSYSTEM DECOMPOSITION DIAGRAM

Detailed Functional Processes, Internal Component Interactions & Data Stores

The Level 1 UML Diagram decomposes Process 0.0 into its six constituent subsystem components and two primary data repositories, detailing how data flows across validation, authentication, order intake, and offline synchronization.



Level 1 Subsystem Descriptions:

- **Process 1.0 & 2.0 (Storefront & Checkout):** Manages real-time catalog rendering, nutritional overlays, cart mutations, and strict Zod validation on shipping details and credit card numbers.
- **Process 3.0 & 4.0 (Authentication & Management):** Gatekeeps administrative capabilities with HMAC SHA-256 tokens and provides product CRUD, 50MB Base64 device upload parsing, and pricing management.
- **Process 5.0 & 6.0 (Ingestion & Offline Sync):** Ingests customer transactions, persists records to MongoDB Atlas (D1), caches to LocalStorage (D2), and triggers automatic batch synchronization upon network reconnect.

7. TECHNICAL ARCHITECTURE & QUICK REFERENCE

Platform Matrix, Endpoints & Security Credentials

LAYER / COMPONENT	TECHNOLOGY STACK	PURPOSE & ROLE
Frontend Framework	React 19 + React Router 7	Client-side SPA UI rendering and routing
Build Engine	Vite 8	Development server & production bundling
Backend Server	Node.js 22 + Express 5	Unified REST API & static asset serving
Database Layer	MongoDB 6+ & Mongoose 9	Document storage with in-memory failover
Form Validation	React Hook Form 7 + Zod 4	Strict schema validation on checkout & forms
Image Processing	FileReader API + 50MB Buffer	Local photo upload & Base64 encoding
API Security	HMAC-SHA256 Bearer Token	Protects sensitive CRUD, sync, and orders routes
Deployment Config	vercel.json SPA Rewrites	Prevents 404 errors on browser page refreshes

QUICK REFERENCE & CREDENTIALS

- Admin URL: /admin | Storefront URL: /

- Admin Credentials: Username: admin | Password: admin (Customizable by owner)